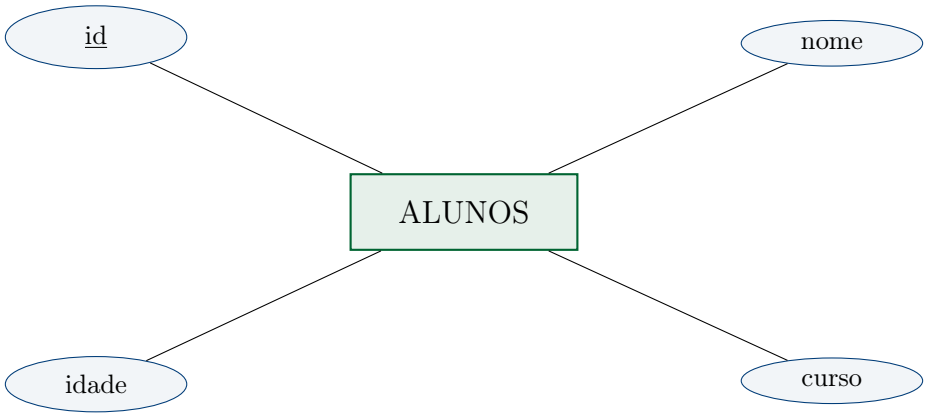
1 Criação da Tabela Alunos

Tabela Alunos

Será construída a tabela `alunos` com as colunas:

- `id` - identificador único (chave primária)
- `nome` - nome do aluno
- `idade` - idade do aluno
- `curso` - curso em que o aluno está matriculado



Registros para Inserção

A tabela a seguir apresenta 20 registros de exemplo a serem inseridos na tabela `alunos`.

id	nome	idade	curso
1	William	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
4	José	18	Letras
5	Ana	20	Medicina
6	Carlos	25	Agronomia
7	Beatriz	19	Psicologia
8	Fernando	21	Arquitetura
9	Helena	23	Engenharia Civil
10	Ricardo	24	Economia
11	Juliana	20	Pedagogia
12	Marcos	22	Odontologia
13	Fernanda	19	Veterinária
14	Gabriel	21	Fisioterapia
15	Larissa	23	Nutrição
16	Roberto	26	Administração
17	Patrícia	20	Enfermagem
18	Samuel	22	Ciência da Computação
19	Amanda	19	Publicidade
20	Igor	21	Educação Física

Comando SQL — Definição e Inserção

```sql
-- Remocao e criacao da tabela
DROP TABLE IF EXISTS Alunos;
CREATE TABLE Alunos(
  id INTEGER PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  idade INTEGER NOT NULL,
  curso VARCHAR(255) NOT NULL
);

-- Insercao dos 20 registros
INSERT INTO Alunos (nome, idade, curso) VALUES
('William', 30, 'Engenharia de Software'),
('Joao', 18, 'Engenharia de Software'),
('Maria', 22, 'Direito'),
('Jose', 18, 'Letras'),
('Ana', 20, 'Medicina'),
('Carlos', 25, 'Agronomia'),
('Beatriz', 19, 'Psicologia'),
('Fernando', 21, 'Arquitetura'),
('Helena', 23, 'Engenharia Civil'),
('Ricardo', 24, 'Economia'),
('Juliana', 20, 'Pedagogia'),
('Marcos', 22, 'Odontologia'),
('Fernanda', 19, 'Veterinaria'),
('Gabriel', 21, 'Fisioterapia'),
('Larissa', 23, 'Nutricao'),
('Roberto', 26, 'Administracao'),
('Patricia', 20, 'Enfermagem'),
('Samuel', 22, 'Ciencia da Computacao'),
('Amanda', 19, 'Publicidade'),
('Igor', 21, 'Educacao Fisica');
```

2 Atualização de Dados (UPDATE)

O comando `UPDATE` é utilizado para modificar registros já existentes em uma tabela. Diferente do `INSERT`, que adiciona novas linhas, o `UPDATE` altera os valores de colunas em linhas que já constam no banco de dados.

Sintaxe e Exemplo

A sintaxe básica envolve definir qual tabela será alterada, quais colunas receberão novos valores (`SET`) e qual condição deve ser atendida para que a alteração ocorra (`WHERE`). Exemplo de atualização da idade da aluna Maria:

```
UPDATE Alunos
SET idade = 20
WHERE nome = 'Maria';
```

Outros Exemplos Práticos

Abaixo estão mais 6 variações do comando `UPDATE` para diferentes cenários:

```
-- 1. Alterar o curso de um aluno especifico pelo ID
UPDATE Alunos SET curso = 'Sistemas de Informacao' WHERE id = 5;

-- 2. Atualizar multiplas colunas ao mesmo tempo
UPDATE Alunos SET nome = 'William Silva', idade = 31 WHERE id =
    1;

-- 3. Alterar a idade de todos os alunos de Engenharia
UPDATE Alunos SET idade = 19 WHERE curso LIKE 'Engenharia%';

-- 4. Incrementar a idade de todos os alunos (Aniversario)
UPDATE Alunos SET idade = idade + 1;

-- 5. Corrigir o nome de um curso para todos os registros
UPDATE Alunos SET curso = 'Direito Penal' WHERE curso =
    'Direito';

-- 6. Atualizar registros em um intervalo de IDs
UPDATE Alunos SET idade = 21 WHERE id BETWEEN 10 AND 15;
```

Atenção

É fundamental utilizar a cláusula `WHERE` em comandos `UPDATE`. Caso ela seja omitida, o banco de dados aplicará a alteração em **todos** os registros da tabela, o que pode causar perda acidental de dados.

3 Boas Práticas: Idade vs. Data de Nascimento

Em projetos de banco de dados reais, **não é recomendado** armazenar a idade do usuário diretamente. Isso ocorre porque a idade é um dado volátil (muda a cada ano), o que exigiria atualizações constantes no banco de dados.

A melhor prática é armazenar a **data de nascimento** e calcular a idade dinamicamente sempre que necessário.

Alterando a Estrutura (ALTER TABLE)

Para corrigir nossa tabela, primeiro devemos remover a coluna `idade` e adicionar a coluna `data_nascimento`.

```
-- 1. Remover a coluna obsoleta
ALTER TABLE Alunos DROP COLUMN idade;

-- 2. Adicionar a coluna de data de nascimento
ALTER TABLE Alunos ADD COLUMN data_nascimento DATE;
```

Preenchendo Dados Aleatórios

Como removemos a coluna de idade e adicionamos a de nascimento, os registros atuais estão com a nova coluna vazia. Podemos usar uma técnica avançada do SQLite para gerar datas de nascimento aleatórias para teste.

```
UPDATE Alunos
SET data_nascimento = date(
    '1970-01-01',
    (abs(random()) % (julianday('2025-12-31') -
       julianday('1970-01-01'))) || ' days'
);
```

Nesse comando, usamos `julianday` para converter datas em números de dias, calculamos um intervalo aleatório e somamos à data base.

Calculando a Idade na Consulta (SELECT)

Para exibir a idade sem armazená-la, realizamos o cálculo diretamente no `SELECT`. A fórmula abaixo é precisa, pois subtrai 1 ano caso o aluno ainda não tenha feito aniversário no ano corrente.

```
SELECT
  id,
  nome,
  curso,
  data_nascimento,
  (STRFTIME('%Y', 'now') - STRFTIME('%Y', data_nascimento)) -
  (STRFTIME('%m-%d', 'now') < STRFTIME('%m-%d',
     data_nascimento)) AS idade
FROM Alunos;
```

Dica Prática

A expressão `(data_atual < data_aniversario)` retorna 1 (verdadeiro) ou 0 (falso) no SQLite, permitindo esse ajuste automático de forma elegante.

4 Análise de Normalização e Design de Dados

Abaixo, analisaremos como o design da nossa tabela se comporta ao lidar com relacionamentos do mundo real.

Cenário: O Problema da Duplicação

O problema surge quando tentamos representar que um aluno está matriculado em mais de um curso ao mesmo tempo.
Conceitualmente, temos a relação: **Ana → {Engenharia de Computação, Direito}**.
Ao inserirmos essa informação na estrutura atual:

```
INSERT INTO Alunos (nome, curso, data_nascimento)
VALUES ('Ana', 'Direito', '2021-07-27');
```

O resultado no banco gera uma duplicação de toda a entidade "Ana":

id	nome	curso	data_nascimento
5	Ana	Engenharia de Computação	2021-07-27
13	Ana	Direito	2021-07-27

Essa duplicação de linhas é um sinal claro de que a tabela não está corretamente normalizada para o domínio, pois mistura dados do *Aluno* com dados da *Matrícula*.

Solução: Modelagem Correta (Separação de Entidades)

Para corrigir esse problema e eliminar a redundância, devemos separar as informações em duas tabelas distintas:

1. Tabela Alunos (id, nome, data_nascimento)

id	nome	data_nascimento
5	Ana	2021-07-27

2. Tabela Aluno_Curso (id_aluno, curso)

id_aluno	curso
5	Engenharia de Computação
5	Direito

Atenção

Ao normalizar dessa forma, cada informação é armazenada em apenas um lugar. Se precisarmos alterar a data de nascimento da Ana, faremos isso em apenas um registro na tabela `Alunos`, sem risco de inconsistências.

5 Migração de Dados: Criando a Tabela Curso

Para implementar a modelagem correta que discutimos, o primeiro passo é extrair os nomes dos cursos que já existem na tabela `Alunos` para uma nova tabela dedicada.

Identificando Cursos Únicos

Podemos usar a cláusula `GROUP BY` para listar todos os cursos distintos que temos atualmente:

```sql
SELECT curso FROM Alunos GROUP BY curso;
```

O resultado será uma lista única que servirá de base para nossa nova tabela:

curso
Direito
Engenharia de Computação
Letras
Medicina

Criando e Populando a Tabela Curso

Agora, criamos a tabela `Curso` e utilizamos o comando `INSERT INTO ... SELECT` para migrar os dados automaticamente.

```sql
-- 1. Criar a nova tabela
CREATE TABLE Curso (
  id INTEGER PRIMARY KEY,
  nome VARCHAR
);

-- 2. Migrar os dados da tabela Alunos para Curso
INSERT INTO Curso (nome)
SELECT curso FROM Alunos GROUP BY curso;
```

Dica Prática

O uso de `INSERT INTO ... SELECT` é uma técnica poderosa para reestruturar bancos de dados existentes, permitindo que você popule novas tabelas diretamente a partir de consultas em tabelas antigas.

6 Relacionando as Tabelas (JOIN)

Com a tabela `Curso` criada e populada, o próximo passo é descobrir qual o ID numérico correspondente para cada curso textual que está na tabela `Alunos`. Para isso, utilizamos o comando `JOIN`.

Consulta de Relacionamento

A consulta abaixo une as duas tabelas comparando o nome do curso na tabela original com o nome na nova tabela de cursos.

```
SELECT Alunos.id, Curso.id
FROM Alunos
JOIN Curso ON Alunos.curso = Curso.nome;
```

Resultado do Mapeamento

O resultado desta consulta nos mostra a relação entre o ID do aluno e o novo ID do curso:

id (Aluno)	id (Curso)
1	2
2	2
4	3
5	2
6	1
7	2
8	4
9	1
10	2
11	4
12	2
13	1

7 Finalizando a Normalização: A Tabela Associativa

O passo final para resolver o problema da redundância e da má modelagem é a criação de uma tabela associativa (ou tabela de ligação). Esta tabela ligará os alunos aos seus respectivos cursos através de chaves estrangeiras.

Criando a Tabela AlunoCurso

A tabela `AlunoCurso` armazena apenas IDs, referenciando as chaves primárias das tabelas `Alunos` e `Curso`.

```
CREATE TABLE AlunoCurso (
  id INTEGER PRIMARY KEY,
  aluno_id INTEGER,
  curso_id INTEGER,
  FOREIGN KEY (aluno_id) REFERENCES Alunos(id),
  FOREIGN KEY (curso_id) REFERENCES Curso(id)
);
```

Populando a Tabela Associativa

Agora, utilizamos a mesma lógica de `JOIN` que testamos anteriormente para inserir os dados de forma automatizada:

```
INSERT INTO AlunoCurso (aluno_id, curso_id)
SELECT Alunos.id, Curso.id
FROM Alunos
JOIN Curso ON Alunos.curso = Curso.nome;
```

Verificação Final

Para conferir o resultado da nossa normalização, basta consultar a nova tabela:

```
SELECT * FROM AlunoCurso;
```

Atenção

Agora que os dados estão na tabela `AlunoCurso`, a coluna `curso` na tabela original `Alunos` tornou-se obsoleta e poderia ser removida para completar o processo de limpeza do banco de dados.

8 Refatoração de Esquema: O Padrão Rename-Create-Insert

Em bancos de dados como o SQLite, algumas alterações estruturais (como adicionar chaves estrangeiras a colunas existentes ou remover colunas antigas) não podem ser feitas diretamente com um simples `ALTER TABLE`. Para isso, utilizamos um padrão profissional de refatoração.

Passo 1: Atualização com Subquery Correlacionada

Antes de mudar a estrutura, precisamos preencher a nova coluna `curso_id` com os valores corretos baseados nos nomes dos cursos.

```
UPDATE Alunos
SET curso_id = (SELECT Curso.id FROM Curso WHERE Curso.nome =
    Alunos.curso)
WHERE curso_id IS NULL;
```

O que ocorre aqui? Para cada linha da tabela `Alunos`, o banco executa uma pequena consulta interna na tabela `Curso`, encontra o ID que tem o mesmo nome do curso e "traz"esse número para a coluna `curso_id`.

Passo 2: Reconstrução da Tabela (O Ciclo de Refatoração)

Agora que temos os IDs, vamos oficializar a nova estrutura. Como o SQLite não permite adicionar restrições de `FOREIGN KEY` em tabelas já criadas, seguimos estes 4 passos:

```
-- 1. Renomear a tabela original para manter os dados seguros
ALTER TABLE Alunos RENAME TO Alunos_old;

-- 2. Criar a nova tabela Alunos com o esquema perfeito
CREATE TABLE Alunos (
  id INTEGER PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  data_nascimento DATE,
  curso_id INTEGER,
  FOREIGN KEY (curso_id) REFERENCES Curso(id)
);

-- 3. Migrar os dados da tabela temporaria para a nova
INSERT INTO Alunos (id, nome, data_nascimento, curso_id)
SELECT id, nome, data_nascimento, curso_id FROM Alunos_old;

-- 4. Excluir a tabela antiga
DROP TABLE Alunos_old;
```

Verificação do Novo Esquema

Agora, ao consultar a tabela, veremos que ela está limpa e devidamente relacionada por IDs, não mais por textos:

```
SELECT * FROM Alunos;
```

Dica Prática

Este processo de "Rename-Create-Insert-Drop"é a maneira padrão e mais segura de realizar mudanças profundas em esquemas de bancos de dados sem perder dados existentes.

9 Consultas Avançadas: Múltiplos JOINs e Aliases

Em um banco de dados normalizado, as informações costumam estar espalhadas por várias tabelas. Para reuni-las em um único relatório, utilizamos múltiplos comandos `JOIN` em sequência.

Chaveando com a Tabela Associativa

Nesta consulta, ligamos a tabela `Alunos` à tabela `AlunoCurso` e, em seguida, ligamos o resultado à tabela `Curso`. Também utilizamos **Aliases** (`AS`) para dar apelidos curtos às tabelas e nomes amigáveis às colunas.

```
SELECT
  A.id,
  A.nome AS NomeDoAluno,
  A.data_nascimento,
  C.nome AS NomeDoCurso
FROM Alunos AS A
JOIN AlunoCurso AS AC ON A.id = AC.aluno_id
JOIN Curso AS C ON AC.curso_id = C.id;
```

Resultado Final Consolidado

O resultado abaixo mostra como o banco de dados reuniu os dados pessoais da primeira tabela com os nomes dos cursos da terceira tabela, passando pela tabela de ligação:

id	NomeDoAluno	data_nascimento	NomeDoCurso
1	William	2011-10-09	Engenharia de Computação
2	João	1971-04-24	Engenharia de Computação
4	José	2023-07-30	Letras
5	Ana	1985-05-31	Engenharia de Computação
6	Pedro	1988-01-06	Direito
7	Carla	1999-01-23	Engenharia de Computação
8	Lucas	1971-01-29	Medicina
9	Sofia	2000-03-02	Direito
10	Gabriel	2024-12-04	Engenharia de Computação
11	Isabella	1986-08-25	Medicina
12	Rafael	1976-08-07	Engenharia de Computação
13	Ana	2021-07-27	Direito

Dica Prática

Encadear múltiplos `JOINs` é a base de quase todos os relatórios complexos em sistemas reais. O uso de apelidos (`A`, `AC`, `C`) é essencial para manter o código legível e evitar confusão entre colunas de nomes iguais.